



Faculty of Engineering

Internal Design Document

STM32 Firmware Design for Distributed Real-Time Telemetry System for QUT Microgrid

Date: April 29, 2026
Student Engineers: Dylan Hessing & Ryan Bock
Student IDs: n11245310
Supervisor: Assoc. Prof. Geoff Walker

Version History

Version	Date	Author	Changes / Comments
1	April 29, 2026	Dylan Hessing	Initial Design

Contents

1	FreeRTOS Adoption: Design Rationale	2
1.1	Motivation	2
1.2	Task Decomposition Over Superloop Architecture	2
1.3	ADC Sampling Integrity	2
1.4	Development Velocity and Ecosystem Maturity	3
1.5	Summary	3

1 FreeRTOS Adoption: Design Rationale

1.1 Motivation

The DAQ firmware must concurrently manage several distinct real-time responsibilities: high-rate ADC sampling, PTP clock synchronisation (IEEE 1588), MQTT telemetry publication, and peripheral management (Ethernet, SPI/I²C). Implementing these as a monolithic superloop introduces structural problems: timing jitter between subsystems, polling overhead competing with acquisition, and brittle sequencing logic that degrades maintainability as complexity grows.

FreeRTOS is selected as the firmware RTOS to address these concerns.

1.2 Task Decomposition Over Superloop Architecture

A bare-metal superloop executes all responsibilities sequentially within a single loop iteration. Any blocking operation (waiting for an Ethernet ACK, a PTP timestamp correction, or an MQTT broker handshake) directly stalls the entire loop, introducing indeterminate latency across all subsystems.

FreeRTOS enables preemptive multitasking: each logical responsibility is encapsulated as an independent task with its own stack and priority assignment.

Table 1: FreeRTOS Task Allocation

Task	Priority	Responsibility
ADC_SampleTask	High	Triggered acquisition, DMA management
PTP_SyncTask	High	IEEE 1588 clock discipline
MQTT_PublishTask	Normal	Packetising and publishing telemetry

This decomposition maps cleanly onto the system's logical architecture and makes the firmware structure self-documenting. Inter-task dependencies are expressed explicitly through FreeRTOS queue and semaphore APIs, rather than implicitly through shared globals and flag polling.

1.3 ADC Sampling Integrity

A common concern when introducing an RTOS is scheduler-induced jitter disrupting deterministic sampling. This does not apply here because ADC acquisition is decoupled from the scheduler via DMA and hardware timers.

The STM32 ADC is configured in timer-triggered DMA mode:

- A hardware timer (e.g. TIM2) fires at a precise, crystal-referenced interval; entirely independent of the FreeRTOS tick.
- Each timer event triggers an ADC conversion; the result is transferred to a memory buffer by DMA with zero CPU involvement.
- ADC_SampleTask wakes only when DMA signals a buffer-full condition (via `xTaskNotifyFromISR` or a binary semaphore), consuming completed samples rather than performing acquisition itself.

The scheduler therefore has no influence over the ADC trigger cadence. Sampling jitter is determined by the hardware timer and ADC clock, not by task preemption.

1.4 Development Velocity and Ecosystem Maturity

FreeRTOS is the dominant RTOS in the STM32 ecosystem. ST's HAL and CubeMX toolchain provide first-party FreeRTOS integration via the CMSIS-RTOS v2 wrapper, including pre-validated configurations for the STM32 device family. Direct project benefits include:

- **Synchronisation primitives:** mutexes, queues, and semaphores cover all inter-task communication needs (e.g. passing sample buffers from `ADC_SampleTask` to `MQTT_PublishTask`) without bespoke implementation.
- **Network stack integration:** LwIP, the embedded TCP/IP stack used for MQTT over Ethernet, has a documented FreeRTOS threading model with known-good integration patterns.
- **PTP driver compatibility:** IEEE 1588 hardware timestamp units on STM32 are accessed via HAL; the FreeRTOS task model integrates naturally with the interrupt-driven PTP servo loop.
- **Reduced driver development burden:** ST-provided middleware (ETH HAL, LwIP, FreeRTOS port) is tested and documented, avoiding hand-rolled scheduling, stack management, or network primitives.

This materially reduces implementation risk and accelerates development relative to a bare-metal approach.

1.5 Summary

FreeRTOS is adopted because the DAQ node's responsibilities are genuinely concurrent, and the RTOS provides the correct abstraction for managing that concurrency safely and legibly. It introduces no risk to ADC sampling fidelity because acquisition is hardware-driven and DMA-managed, making it scheduler-independent by design.